

[← Portfolio](#)

PROJECT 1: COLORIZING THE PROKUDIN-GORSKII PHOTO COLLECTION

We're hoping to take the digitized Prokudin-Gorskii glass plate images and align the blue, red, & green channels to form a single color image. It's an interesting task; we need some method of determining the best way to align these images that scales as images get large.

Single-scale alignment

Naively, we can just search over a range of displacements. And this works for the smaller JPG images; they're not that big (a height of around 340 pixels per channel), and through experimentation, we find that generally searching through a displacement of ± 15 leads to good alignment (the maximum shift magnitude is 7 for the three that we test).

Even going through this exercise helps us clarify a few details. One, what does it mean to displace the image? Essentially, we're shifting the image for a given channel left or right, up or down by a few pixels—and checking whether it aligns better with some baseline channel (for example, blue). When we shift, the image is going to be a different size, so we wrap the image around using our friend `np.roll`.

Next, how do we quantify “alignment”? There are two metrics we can use. First is L2 norm, or Euclidean distance, given by:

$$L_2(\text{img}_1, \text{img}_2) = \sqrt{\sum_{(i,j) \in \Omega} \left(\text{img}_1(i,j) - \text{img}_2(i,j) \right)^2}$$

where Ω is the set of pixels we use to align the image. Intuitively, we're just treating each of the images as a 1-D vector (for all pixels in Ω) and finding the distance between the two vectors.

But if there are lighting or brightness inconsistencies between two channels or even within a channel, those are magnified. That means we might not converge to the “optimal” displacement due to simple irregularities in the image. In contrast, Normalized Cross-Correlation (NCC)—intuitively just a dot product between normalized vectors—takes care of these issues by normalizing each image and calculating similarity. Mathematically, it’s given by

$$\text{NCC}(\text{img}_1, \text{img}_2) = \frac{\sum_{(i,j) \in \Omega} (\text{img}_1(i,j) - \mu_1)(\text{img}_2(i,j) - \mu_2)}{\|\text{img}_1 - \mu_1\|_2 \|\text{img}_2 - \mu_2\|_2}$$

where μ_1 and μ_2 are the mean pixel value over Ω for images 1 and 2 respectively. Our goal is for the best alignment: minimizing L2 norm or maximizing NCC. We formulate this optimization search in the code as a maximization problem, and thus maximize the negative L2 norm. However, NCC, in its invariance to per-channel brightness, is more reliable and thus used to align the final images.

And then there’s the border question; the edges of these images are super noisy, and they’re not super consistent. We find that all of the raw images have some white border at the top and bottom that splitting the long image into three channels (by merely cutting it into thirds, per the starter code provided) leads to inconsistencies in per-channel whitespace. To remedy this, we crop the first and last 0.5% of the image before splitting it into channels. Separately, we set Ω to be the middle 80% of the image (cropping the top/bottom/left/right 10%) and compute the quantities above (NCC/L2) on that subset, so we only align these images based on their interior (which is what carries the signal).

Single-scale results

Here are the results on the three single scale images.

We provide the displacements as (x, y) coordinates, representing the horizontal and vertical pixel shifts from the baseline (Green is aligned against Blue, Red is aligned against the Aligned Green—more details on this below). No defects to see here :)



CATHEDRAL · G (2, 1) · R (3, 4)



MONASTERY · G (2, -7) · R (3, -5)



TOBOLSK · G (3, -1) · R (4, -1)

Multi-scale alignment

When the images get larger, we have to construct an image pyramid. We continually downsample the image, reducing its height and width by factors of 2 (floor division) until we reach the “smallest” image (determined by me as having a height < 500 pixels, corresponding to the size of our small JPGs for single-scale alignment). To implement this, I use `skimage.transform.resize`—with the anti-aliasing flag turned on. Under the hood, it essentially creates a Gaussian kernel with the appropriate standard deviation for our downscaling factor,

and applies the blur to create a coarser image. In doing this low-pass filtering, we mitigate the aliasing and misrepresentation effects that naive resizing would cause.

Once we reach this base image, we're able to do an exhaustive search over a big window of displacements. Then, we can find the best displacement on the coarser image; if the displacement is (x, y) , that corresponds to $(2x, 2y)$ on the scaled-up image. Then we have a rough idea of how much we need to shift, but we obviously want a finer representation; however, the whole point of this resampling mechanism is to ensure that we don't have to search a big window of displacements each time. So for these larger images, we start with searching a displacement size of ± 2 on the largest image, and scale that by 2 every time we resize the image by a factor of $\frac{1}{2}$. That way, on the smallest image, we search a wide range of displacements, and as the image gets larger, we search smaller and smaller ranges. This proved to increase efficiency without compromising results. We implement this recursively: resizing until we reach a base case, then aligning, and then feeding the new candidate displacements and displacement sizes upstream—until we get the final displacement number on our original image. Note that to standardize our work with JPGs and TIFs, we converted the image data type to floats; then when saving to JPG, I convert back to an 8-bit unsigned integer format.

Finally, the starter code has us align both the green and red images to the blue image. This doesn't work perfectly, particularly for the emir; there's clear misalignment of the red channel relative to the other two. However, if you align the green image to the blue, and then the red image to the aligned version of the green image, the artifacts go away—and the emir is much better aligned. I hypothesize this is just due to the fact that red and green are closer in wavelength than red and blue; so there are likely to be more similarities on key image features, making the alignment less noisy and easier to find.

Note that the multi-scale alignment for the three single-scale images was essentially the same, as they directly hit the base case. I thus show the results for the other 11 TIFs below.

Multi-scale results

Here are the multi-scale result alignments.



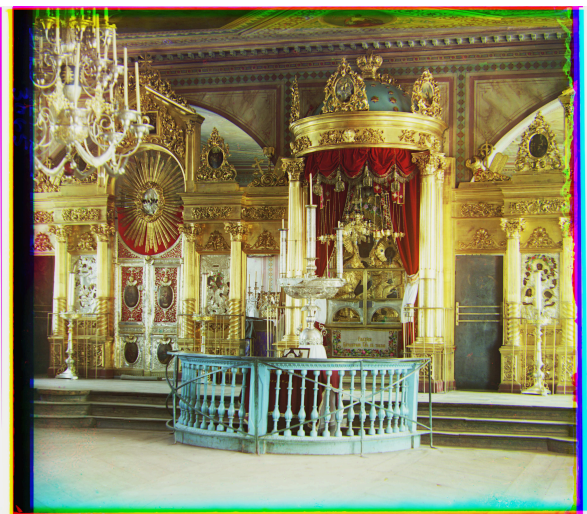
CHURCH · G (4, -7) · R (-4, -6)



EMIR · G (24, 16) · R (41, 40)



HARVESTERS · G (17, 28) · R (14, 61)



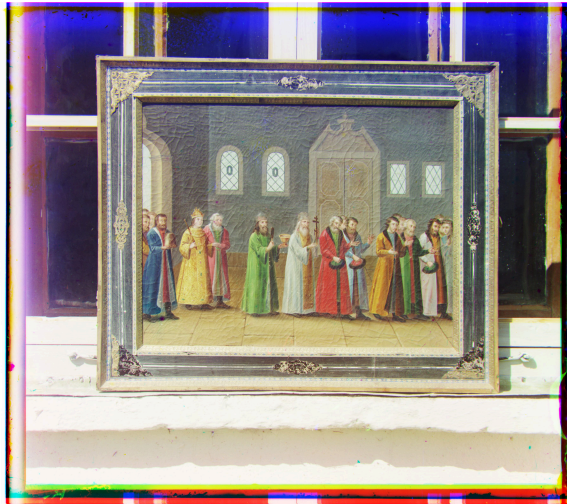
ICON · G (17, 8) · R (22, 23)



ILEMSELGA · G (7, 7) · R (11, 64)



MELONS · G (10, 50) · R (14, 114)



RELIGIOUS PAINTING · G (3, -4) · R (7, 5)



SELF PORTRAIT · G (29, 47) · R (37, 113)



SIREN · G (-6, 17) · R (-24, 32)



THREE GENERATIONS · G (14, 21) · R (11, 47)



WHARF · G (-7, -17) · R (-16, 18)

Three of my own picks

*I chose three images from the library that I called meadow, monument, and mosque.
The results are below.*



MEADOW · G (12, -2) · R (6, 12)



MONUMENT · G (6, -4) · R (13, 40)



MOSQUE · G (35, 23) · R (62, 62)

Reflection

Overall, none of the images seem to have large alignment defect issues. Nevertheless, there are still minor imperfections. It's important to note the noisiness of the borders and edges across all of these images, a product of our imperfect cropping heuristics. In addition, many of the

people in the back of the harvesters image are not perfectly aligned, likely due to motion in the image. The same is true with the red glow off of the beard of the left man in three generations. And there are blotches of color in these images (see the green square in self-portrait, and the more noisy dots in melons). These are likely imperfections and noise within each color channel that—while not impeding alignment—result in imperfect colorization.